



Discrete Optimization

An efficient genetic algorithm to solve the resource-constrained project scheduling problem with transfer times: The single mode case



Roubila Lilia Kadri, Fayez F. Bector

Centre Interuniversitaire de Recherche sur le Réseau d'Entreprises, la Logistique et le Transport (CIRRELT), Faculté des sciences de l'administration, Université Laval, Quebec, QC, Canada

ARTICLE INFO

Article history:

Received 6 April 2016

Accepted 8 July 2017

Available online 12 July 2017

Keywords:

Project scheduling

Resource transfer time

Genetic algorithms

ABSTRACT

In this article, we address the resource-constrained project scheduling problem with transfer times (RCP-SPTT). We assume that pre-emption is not allowed, and precedence relations are zero-lag finish-to-start relations. We also assume that the durations and resource transfer times of activities are known and deterministic. The objective is to choose a start time for each activity of the project so that the project duration is minimized, while satisfying precedence relations, resource availabilities, and resource-transfer time constraints. We propose a new genetic algorithm using a two-point crossover operator. The experiment, conducted on a large number of instances, shows that the proposed algorithm performs better than several solution methods previously published.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

The standard version of the resource-constrained, project scheduling problem (RCPS) consists of scheduling a set of activities ($j = 1, \dots, J$) linked by zero-lag, and finish-to-start precedence relations, while pre-emption is not allowed. The resources required to perform activities are renewable and available in limited quantities in each period. Each activity has a unique and known duration, and uses a constant amount of resources for each period of its execution. The objective of the RCPS is to minimize the duration of the project (make span) while respecting precedence- and resource-constraints. This problem is known to be NP-hard (Blazewicz, Lenstra, & Rinnooy Kan, 1983).

To solve the RCPS optimally, various branch- and bound-procedures have been developed (Brucker, Knust, Schoo, & Thiele, 1998; Carlier & Latapie, 1991; Demeulemeester & Herroelen, 1996; Patterson, Talbot, Slowinski, & Wegelaz, 1990). However, none of these techniques are capable of solving large instances of the problem in a reasonable computational time.

Hence, several researchers have proposed heuristic methods to solve the RCPS. The majority of these methods fall into two categories: priority rule-based heuristics, and neighborhood search heuristics. The former are construction heuristics which start with an empty schedule and add activities one-by-one until all activities are scheduled. This operation is controlled by a scheduling-generation scheme and a priority rule. Two different

scheduling schemes, serial and parallel (Brooks & White, 1965; Kelley, 1963), are widely used to develop most heuristic techniques published in the literature (Bector, 1990; Kolisch, 1996a; Kolisch & Drexel, 1997). In order to improve a feasible schedule obtained by a construction heuristic, various neighborhood search methods have been developed: simulated annealing (Bector, 1996; Bouleimen & Lecocq, 2003), tabu search (Baar, Brucker, & Knust, 1999; Nonobe & Ibaraki, 2002), scatter search (Debels, De Reyck, Leus, & Vanhoucke, 2006; Mobini, Rabbani, Amalnik, Razmi, & Rahimi-Vahed, 2009), and the genetic algorithm (Alcaraz, Maroto, & Ruiz, 2003; Hartmann, 2001; Mendes, Goncalves, & Resende, 2009; Valls, Ballestin, & Quintanilla, 2004; Zamani, 2013).

The standard RCPS, as defined above, assumes that no transfer times are needed to transfer resources between the sites of execution of different tasks. In practice, however, it happens that moving resources between these sites may take a significant amount of time. For example, heavy construction equipment might be requested at different construction sites, and moving them from one site to another may require several hours.

In such cases, modeling and solving the project scheduling problem without taking into consideration transfer times, could produce unachievable schedules. Recently, Krüger and Scholl (2009) developed methods to solve the resource-constrained project scheduling problem with transfer times (RCP-SPTT) for the single- and multi-project cases. They formulated both problems as integer-linear models, and developed a priority rule-based heuristic-solution approach. Krüger (2009) developed several adaptations of the genetic algorithm (GA) for this problem, and recently, a flow-based tabu search algorithm was developed by

E-mail address: fayez.bector@fsa.ulaval.ca (F.F. Bector).

Poppenborg and Knust (2014 and 2016). We propose a new and efficient GA to solve this problem.

The remainder of this paper is organized as follows: Section 2 describes the RCPSPPT, and Section 3 presents a mathematical formulation for it; Section 4 describes the proposed GA, and Section 5 reports on and discusses the results of the conducted evaluation, and Section 6 concludes the paper, and looks at possible follow-ons.

2. Problem description

In the RCPSPPT, we have a set of activities, J , a set of zero-lag, finish-to-start precedence relationships between these activities, P , a set of renewable resources, K , and the limits on the resource availability are denoted Q_k ; $\forall k \in K$. Each activity i has a set of immediate predecessors, $P_i \subseteq J$, a set of immediate- and indirect-predecessors, $\pi_i \subseteq J$, a set of immediate successors, $Z_i \subseteq J$, and a set of immediate and indirect successors, $\sigma_i \subseteq J$. Each activity has a unique and known duration d_i , and uses a constant amount of resources, q_{ik} ; $\forall k \in K$, for each period of its execution. We also define $I_i = [EST_i, LST_i]$, the time window between the earliest and the latest start times of activity i . These times are calculated using the critical path method, while using feasible project durations, denoted by T . We assume that the transfer of a unit or several units of renewable resource k from the execution location of activity i to the execution location of activity j requires a transfer time denoted Δ_{ijk} . All transfer times are assumed to fulfill the triangular inequality; i.e., $\Delta_{ijk} \leq \Delta_{ilk} + \Delta_{ljk}$, $\forall i, j, l \in J$.

Without loss of generality, we assume that the project includes a dummy start activity (activity 1) and a dummy finish activity (activity F). These two activities have a zero duration and require the entire available amount Q_k of every resource k ; (i.e., $q_{1k} = q_{Fk} = Q_k$; $\forall k \in K$). This is because the dummy start activity should provide resources to other activities, while the dummy finish activity should collect them back. Transfer times from the dummy start activity, and to the dummy finish activity are not necessarily nil and can have any value. The objective in RCPSPPT is to construct a schedule that minimizes the project duration, while satisfying precedence, resource, and transfer-time constraints.

Fig. 1 shows an example of a project comprising nine activities to be scheduled using one resource type with eight available units. The transfer time matrix Δ_{ij} is also shown in Fig. 1. The optimal schedule of this example with a make span of 13 time-periods is shown in Fig. 2, where arrows indicate transfers of resource units.

3. Mathematical formulation

In addition to the notation presented above, we need to use three decision variables. First, we use a binary variable x_{it} , that take the value 1 if, and only if, activity i starts at the beginning of time period t . Second, we use another binary variable, y_{ijk} , that takes the value 1 if, and only if, at least one unit of resource k is transferred from activity i to activity j . Finally, f_{ijk} gives the number of units of resource k to transfer from activity i to activity j .

Using this notation, the mathematical model of the RCPSPPT can be written as follows (see also Krüger & Scholl, 2009):

$$\text{Minimize : } \sum_{t \in I_F} t \cdot x_{Ft} \quad (1)$$

Subject to :

$$\sum_{t \in I_i} x_{it} = 1; \quad \forall i \in J \quad (2)$$

$$\sum_{t \in I_j} t \cdot x_{jt} - \sum_{t \in I_i} (t + d_i) x_{it} \geq 0; \quad \forall (i, j) \in P \quad (3)$$

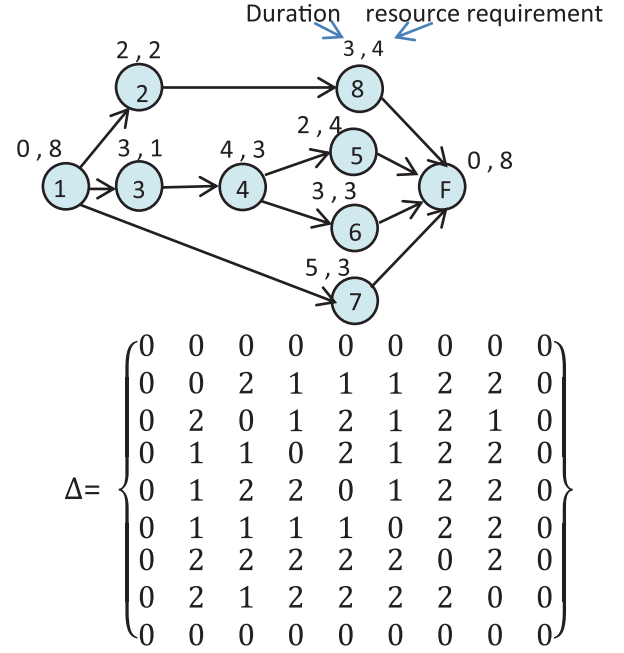


Fig. 1. A RCPSPPT example.

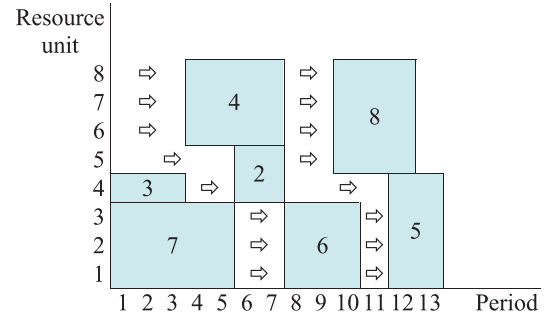


Fig. 2. Optimal solution of the RCPSPPT example.

$$\sum_{t \in I_j} t \cdot x_{jt} - \sum_{t \in I_i} (t + d_i) x_{it} - (T + \Delta_{ijk}) \cdot y_{ijk} \geq -T; \quad \forall i \in J - \{F\}, \quad \forall j \in J - \pi_i, \quad \forall k \in K \quad (4)$$

$$f_{ijk} \leq \min(q_{ik}, q_{jk}) \cdot y_{ijk}; \quad \forall i \in J - \{F\}, \quad \forall j \in J - \pi_i, \quad \forall k \in K \quad (5)$$

$$\sum_{i \in J - \pi_i} f_{ijk} = q_{jk}; \quad \forall j \in J - \{1\}, \quad \forall k \in K \quad (6)$$

$$\sum_{j \in J - \pi_i} f_{ijk} = q_{ik}; \quad \forall i \in J - \{F\}, \quad \forall k \in K \quad (7)$$

$$x_{it} \in \{0, 1\}; \quad \forall i \in J, \quad \forall t \in I_i \quad (8)$$

$$y_{ijk} \in \{0, 1\}; \quad \forall i \in J - \{F\}, \quad j \in J - \pi_i, \quad \forall k \in K \quad (9)$$

$$f_{ijk} \geq 0; \quad \forall i \in J - \{F\}, \quad j \in J - \pi_i, \quad \forall k \in K \quad (10)$$

The objective function (1) minimizes the project duration. Constraints (2) ensure that each activity is executed without pre-emption and starts within the time window between its earliest and latest start times. Constraints (3) are the precedence constraints, and constraints (4) ensure that the transfer time of resource k is taken into consideration when some of its units are transferred from activity i to activity j . Constraints (5) imply that,

```

Generate  $G$  initial solutions using the modified regret-based biased random sampling.
 $N_{solutions} = G$ 
WHILE ( $N_{solutions} \leq L$ ):
    • Generate  $G$  offspring solutions from current population using the proposed TPCO.
    • Apply mutation to some of the generated offspring solutions.
    • Determine the fitness (the make-span) of each offspring solution.
    •  $N_{solutions} = N_{solutions} + G$ 
    • Add the new  $G$  solutions to the previous generation of  $G$  solutions.
    • Select the best  $G$  solutions to become the next generation.
END WHILE
Retain the best obtained solution

```

Fig. 3. Framework of the proposed genetic algorithm.

1	i_1	i_2	...	i_j	...	F	binary	binary
← Precedence feasible activity list →							0 = Serial or 1 = Parallel	0 = Forward or 1 = Backward

Fig. 4. Solution code (representation).

if $y_{ijk} = 0$, then $f_{ijk} = 0$. They also ensure that if $y_{ijk} = 1$, f_{ijk} does not exceed either the amount of k available at i or the amount required for j . Constraints (6) and (7) are the resources-flow conservation constraints. Constraints (8), (9), and (10) indicate the domain of the decision variables of the model.

4. The proposed genetic algorithm

In this section, we describe a new GA to solve the RCPSPTT. It starts with an initial population of G solutions generated by a modified version of the regret-based biased random sampling heuristic. In our computational experiment presented in Section 5, we use $G = 200$. The original version of this heuristic was proposed by Kolisch and Drexel (1997).

The GA then determines the fitness value (the make span) of each individual, randomly selects a number of pairs of individuals (solutions), and uses a two-point crossover operator (TPCO) to generate two new individuals (offspring solutions) from each selected pair of parent solutions. This TPCO is a modification of the crossover operator proposed by Zamani (2013), the magnet-based crossover (MBX) operator. Afterward, a mutation operation is applied to some of the newly generated individuals.

After computing the fitness of each of the new G offspring solutions, we add them to the G solutions of the current generation. We then select the best G among these $2G$ solutions, and place them in the next generation. This process is repeated until the total number of solutions generated by the algorithm reaches a pre-selected number, L . In our computational experiment we used $L = 1000, 2000, 3000, 4000$, and 5000 . The pseudo-code of the algorithm is presented in Fig. 3.

4.1. Solution coding

The GA proposed in this paper uses an extended activity-list representation of solutions, where each solution is represented (coded) by $|J| + 2$ genes as shown in Fig. 4. The first $|J|$ genes are a precedence-feasible activity list (sequence), meaning that each activity should be placed in the list after all its predecessors. After that, two extra binary genes are added, the first one indicating

the scheduling generation scheme to use in order to generate the corresponding feasible schedule. In this work we adapt the serial (S) and parallel (P) scheduling schemes to construct schedules for the RCPSP with transfer times (these two adaptations are presented in Section 4.3). The second binary gene indicates the scheduling direction (forward or backward F/B) to be used while generating the schedule.

4.2. Generation of the initial population

The first step of the GA consists of generating an initial population of G solutions. As mentioned above, each solution comprises a precedence-feasible activity list, a binary indicating the type of scheduling scheme (serial or parallel) to be used to translate the list into a feasible schedule, and another binary indicating the direction (forward or backward) to be followed while building this schedule. For each of the G initial solutions, its precedence-feasible activity list is obtained using a modified version of the regret-based biased sampling heuristic described in Fig. 5 below. Within this heuristic, three priority rules are used: the minimum latest finish time (minLFT), the minimum latest start time (minLST), and the minimum total slack (minSLK) rules. This biased sampling method is a heuristic that iteratively determines the set of eligible activities (whose predecessors are already included in the list under construction), randomly selects one of them and adds it to the activity list. This process is repeated until a complete precedence-feasible list (sequence) is obtained. Afterward, two binaries are randomly generated and added to complete the code representing the solution.

4.3. Transforming a precedence-feasible activity list into a schedule

Once an individual solution (coded by a precedence-feasible activity list, and the two additional binaries as shown in Fig. 4) is generated, we need to construct the corresponding feasible schedule and calculate its fitness value. The fitness value of a solution is given by the duration of the corresponding schedule.

To construct a feasible schedule, we use an adaptation of either the serial- or the parallel-scheduling schemes (see Brooks & White, 1965; Kelley, 1963; Kolisch 1996b; Krüger & Scholl, 2009). Before we present our adaptation of these two schemes, we introduce the following notation:

u_{ijk}	Number of units of resource k transferred from activity i to activity j
v_{ik}	Number of units of k still available at the site of activity i
a_{kt}	Number of units of k still available at time t

```

Initialize the activity list (sequence) :  $\sigma = \{1\}$ 
Randomly choose a priority rule among the three rules (minLFT, minLST, and minSLK)
Let  $S$  be the set of eligible activities;  $S = \{i \in J \mid P_i = \{1\}\}$ 
WHILE  $S \neq \emptyset$ 
    • Determine  $p_j$ ;  $\forall j \in S$ , (for example,  $p_j$  is the LFT if we have chosen the minLFT rule)
    • For each eligible activity  $j$ , calculate  $r_j$ , its regret value, and  $\psi_j$  its selection probability, where:  $r_j = 1 + (\max_{i \in S} p_i) - p_j$  and  $\psi_j = \frac{r_j}{\sum_{i \in S} r_i}$ 
    • Based on the probability  $\psi_j$  and using the simple roulette technique, randomly select an activity among the eligible ones. Let  $l$  be the selected activity. Add  $l$  to the sequence;  $\sigma = \sigma + \{l\}$ 
    • Update  $S$ , the set of eligible activities;  $S = \{i \in J \mid P_i \subseteq \sigma\}$ 
END WHILE

```

Fig. 5. The modified regret-based biased random sampling procedure.

D	Set of activities already scheduled
C_t	Set of activities scheduled to finish by the beginning of t
FT_i	Finish time of activity i
Z_{jk}	Set of activities that can provide units of k to activity j
Γ_{jk}	Set of pair of activities (i, l) such that i provides units of k to l , and that activity j can be placed in-between and uses some of these units of k , before passing them to l without delaying l

4.3.1. The serial scheduling scheme

Fig. 6 presents the forward serial scheduling scheme. The sequence (activity list) representing the solution is an input to this procedure. This iterative procedure sequentially adds activities to the schedule until a complete feasible schedule is obtained. In each iteration, we select the first non-scheduled activity in the sequence, j , and determine its earliest possible start time, t .

All activities which precede activity j in the list, and can deliver at least one unit of resource k to activity j at time t (or earlier), are identified and added to a set denoted Z_{jk} . We also identify all pairs of scheduled activities, i and l , such that activity j can be inserted between them and we can break the resource flow between them to deliver some resource units from i to j , which will later transfer them to l without causing any delay for activity l . All such breakable flows are identified, and the corresponding pair of activities i and l are added to a set denoted Γ_{jk} . Therefore, providing the required units of resource k to j at time t can be either from activities belonging to Z_{jk} , or by breaking some flows between pairs of activities belonging to Γ_{jk} . The maximum amount of resource type k that can be provided to j at period t , s^{jk} , is given by: $s^{jk} = \sum_{i \in Z_{jk}} v_{ik} + \sum_{(i,l) \in \Gamma_{jk}} u_{ilk}$.

If enough resource units are available to schedule activity j at t ; i.e., if $s^{jk} \geq q_{jk}$; $\forall k \in K$, we then schedule activity j to start at the beginning of period t . To provide activity j with resource k , we first use the breakable flows, u_{ilk} , between pairs of activities (i, l) belonging to Γ_{jk} , starting with the pair holding the largest value of u_{ilk} (in case of a tie, activities with the smallest label number will be chosen). If all such flows are used, and more units of k are needed to perform j , we use the units available at the sites of activities i belonging to Z_{jk} , arranged in ascending order of their transfer time Δ_{ijk} (in case of a tie, activity with the smallest label number will be chosen).

Alternatively, if s^{jk} , the transferable amount of resource k , is not sufficient to allow activity j to start at the beginning of time period t , then t is increased until sufficient resources become available. The procedure stops when a complete schedule is constructed.

4.3.2. The parallel scheduling scheme

The parallel scheduling scheme is presented in Fig. 7. It is a straight forward adaptation of the parallel scheduling scheme for the standard RCPSP proposed by Kelley (1963), and by Brooks and White (1965). This iterative procedure has as input the activity list (sequence) that we will transform into a schedule. In each iteration, the procedure identifies the candidate (schedulable) activities, arranges them according to their order in the sequence, and schedules the activity with the highest priority that can be executed with the available amount of resources. It then updates the list of candidate activities, and if the list is not empty, proceeds to schedule activities at the top of the updated list. If the candidate list is empty, but there remain some non-scheduled activities, the procedure goes to the earliest date where certain activities can be scheduled and selects one of them. The procedure ends once all activities are scheduled.

4.4. The proposed two-point crossover operator

The uniform crossover (Bierwirth, Mattfeld, & Kopfer, 1996) and the k -point crossover operators (Djerid, Portman, & Villon, 1996), were suggested to handle scheduling problems. However, the one point crossover (Davis, 1985), and the two-point crossover (Hartmann, 1998) are usually used for solving the standard RCPSP.

Two types of two-point crossover are suggested and, in both types, we start by selecting a contiguous block of genes from one of the parent solutions. In the first type, to create an offspring solution, we modify the sequence of activities within the selected block (Hartmann, 2002; Lova, Tormos, Cervantes, & Barber, 2009), and, in the second type, we keep the block unchanged and modify the sequence of activities outside the block (Zamani, 2013).

The crossover operator suggested by Zamani, MBX, works as follows. First, a contiguous block of genes is selected from the first parent (called the donor parent) by randomly drawing two numbers n_1 and n_2 ($1 < n_1 < n_2 < |J|$), which become the bounding positions in the donor sequence specifying the required contiguous block. Second, it determines the first and last positions of the elements of this block in the sequence of the second parent (called the receiver). These positions in the receiver sequence are denoted n_0 and n_3 , respectively. The offspring sequence is then built by concatenating:

- The activities in positions 1 to $n_0 - 1$ from the sequence of the receiver;
- Followed by the activities in-between positions n_0 and n_3 in the receiver sequence, that are predecessors to any activity in the block;
- Followed by the activities of the block;

```

Initialize:  $D = \{1\}$ 
 $FT_i = 0; \forall i \in J$ 
 $u_{ijk} = 0; \forall i \in J - \{F\}, \forall j \in J - \pi_i$ 
 $v_{1k} = Q_k; \forall k \in K: v_{ik} = 0; \forall i \in J - \{1\}, \forall k \in K$ 
Let  $\sigma$  be the sequence to transform into a schedule
FOR  $\lambda=2$  to  $|J|$ 
  Let  $j$  be the activity in position  $\lambda$  in the sequence  $\sigma$  and  $t = \max_{i \in P_j} FT_i$ 
  WHILE  $j \notin D$ 
    FOR  $k=1$  to  $|K|$ 
       $Z_{jk} = \emptyset$ 
      FOR  $l=1$  to  $\lambda-1$ 
        Let  $i$  be the activity in position  $l$  in the sequence  $\sigma$ 
         $\Gamma_{jk} = \emptyset$ 
        IF  $v_{ik} > 0$  AND  $FT_i + \Delta_{ijk} \leq t$  THEN  $Z_{jk} = Z_{jk} \cup \{i\}$ 
        FOR  $e=1$  to  $\lambda-1$ 
          Let  $m$  be the activity in position  $e$  in the sequence  $\sigma$ 
          IF  $m \neq i$  AND  $u_{imk} > 0$  AND  $FT_i + \Delta_{ijk} \leq t \leq FT_m - d_j - d_m - \Delta_{jmk}$  THEN  $\Gamma_{jk} = \Gamma_{jk} \cup (i, m)$ 
        NEXT  $e$ 
      NEXT  $l$ 
       $s^{jk} = \sum_{i \in Z_{jk}} v_{ik} + \sum_{(i,m) \in \Gamma_{jk}} u_{imk}$ 
    NEXT  $k$ 
    IF  $s^{jk} \geq q_{jk}, \forall k \in K$  THEN
      FOR  $k=1$  to  $|K|$ 
        Arrange all breakable-flow activity pairs  $(i, m) \in \Gamma_{jk}$  in the descending order of  $u_{imk}$ 
        Arrange all potential delivering activities  $i \in Z_{jk}$  in the ascending order of transfer time  $\Delta_{ijk}$ 
         $l = 1$ 
        WHILE  $v_{jk} < q_{jk}$  AND  $l \leq |\Gamma_{jk}|$ 
          Let  $(i, m)$  be the pair in position  $l$  in the set  $\Gamma_{jk}$ 
           $x = \min(u_{imk}, q_{jk} - v_{jk})$ 
           $u_{imk} = u_{imk} - x$ 
           $u_{ijk} = u_{ijk} + x : u_{jmk} = u_{jmk} + x : v_{jk} = v_{jk} + x$ 
           $l = l + 1$ 
        END WHILE
         $e = 1$ 
        WHILE  $v_{jk} < q_{jk}$ 
          Let activity  $i$  be the activity in position  $e$  in the set  $Z_{jk}$ 
           $x = \min(v_{ik}, q_{jk} - v_{jk})$ 
           $v_{ik} = v_{ik} - x$ 
           $u_{ijk} = u_{ijk} + x : v_{jk} = v_{jk} + x$ 
           $e = e + 1$ 
        END WHILE
      NEXT  $k$ 
       $D = D \cup \{j\}$ 
       $FT_j = t + d_j$ 
    ELSE
       $t = t + I$ 
    END IF
  END WHILE
NEXT  $\lambda$ 

```

Fig. 6. Serial scheduling procedure to transform a sequence into a schedule.

- Followed by the activities in positions between n_0 and n_3 in the receiver sequence that are successors to any activity in the block; and
- Followed by the activities in positions between $n_3 + 1$ and $|J|$ in the receiver sequence.

However, some of the activities in the positions between n_0 and n_3 in the receiver sequence may be neither a predecessor nor a successor to any activity in the block. These activities are called **free** activities and are randomly placed, either before or after the block in the offspring sequence. Zamani (2013) suggests that whenever a free activity is encountered, it is placed before the block, with a chance p to overturn this decision. However, once

this decision is overturned, the remaining free activities are placed after the block. If q is the number of free activities, Zamani fixes $p = 0.5$ for $q = 1$, and $p = 2/(q + 2)$ for $q > 1$.

Our proposed two-point crossover operator is a modification of Zamani's operator, and, as such, can be called a modified magnet-based crossover operator. It differs in that free activities are randomly placed either before or after the activities of the block, as long as this does not violate any precedence relation. This is implemented as follows. For every free activity, we determine all possible positions before or after the block, where it can be placed without violating any precedence relation. We then randomly chose one of these positions. This makes our operator more disruptive than Zamani's, as it introduces more diversification in

```

Initialize:   $t = 1$ 
             $C_t = \{1\}$  (set of activities completed by time  $t$ )
             $FT_i = 0; \forall i \in J$ 
             $v_{1k} = Q_k; \forall k \in K; v_{ik} = 0; \forall i \in J - \{1\}, \forall k \in K$ 
             $\sigma = \{1, i_1, i_2, \dots, F\}$  the sequence to transform into a schedule
             $S = \{\forall i \in J \mid P_i = C_t\}$  (set of candidate activities at time  $t$ )
             $D = \{1\}$  (set of scheduled activities)
             $a_{kt} = Q_k; \forall k \in K, t = 1, \dots, T$  (units of  $k$  still available at  $t$ )
            Arrange activities in the list  $S$  according to their order in  $\sigma$ 

WHILE  $|D| < |J|$ 
  WHILE  $S \neq \emptyset$ 
    Let  $j$  be the first activity in the ordered list  $S$ 
    FOR  $k = 1$  to  $|K|$ 
       $Z_{jk} = \emptyset$ 
      FOR  $l = 1$  to  $|C_t|$ 
        Let  $i$  be the activity in position  $l$  in the set  $C_t$ 
        IF  $v_{ik} > 0$  AND  $FT_i + \Delta_{ijk} \leq t$  THEN  $Z_{jk} = Z_{jk} \cup \{i\}$ 
      NEXT  $l$ 
    NEXT  $k$ 
    IF  $\sum_{i \in Z_{jk}} v_{ik} \geq q_{jk}, \forall k \in K$  THEN
      FOR  $k = 1$  to  $|K|$ 
        Arrange activities  $i \in Z_{jk}$  in the ascending order of their transfer time  $\Delta_{ijk}$ 
         $e = 1$ 
        WHILE  $v_{ik} < q_{jk}$ 
          Let activity  $i$  be the activity in position  $e$  in the set  $Z_{jk}$ 
           $x = \min(v_{ik}, q_{jk} - v_{jk}) : v_{ik} = v_{ik} - x : u_{ijk} = u_{ijk} + x : v_{jk} = v_{jk} + x$ 
           $e = e + 1$ 
        END WHILE
      NEXT  $k$ 
       $D = D \cup \{j\}$ 
       $FT_j = t + d_j$ 
      FOR  $\tau = t$  to  $FT_j$ 
        FOR  $k = 1$  to  $|K|$ 
           $a_{kt} = a_{kt} - q_{jk}$ 
        NEXT  $k$ 
      NEXT  $\tau$ 
    END IF
     $S = S - \{j\}$ 
  END WHILE
   $t = t + 1$ 
  Update  $C_t$  the list of activities completed by time  $t$ ;  $C_t = \{\forall j \in J \mid FT_j < t\}$ 
   $S = \{\forall j \in J - D \mid P_j \subseteq C_t; q_{jk} \leq a_{kt}; \forall k \in K\}$  (the list of candidate activities at time  $t$ )
  IF  $S \neq \emptyset$  THEN
    Arrange activities in the list  $S$  according to their order in  $\sigma$ 
  END IF
END WHILE

```

Fig. 7. Parallel scheduling scheme for the RCPSPTT.

the process. Our numerical experiment shows that our operator yields better results.

In our computational experiment presented in Section 5, n_1 was randomly drawn from a uniform distribution between 1 and $0.75 \times |J|$, while $n_2 = n_1 + c$, where $c = 5, 10, 20$, or 25 for the instances with 30, 60, 90, and 120 activities respectively.

To illustrate the application of the proposed two-point crossover operator described here, we use the numerical example presented in Fig. 1. Two feasible solution codes are presented at the top of Fig. 8, and are considered as the donor and receiver parents, respectively. Assuming that we randomly draw $n_1 = 6$ and $n_2 = 8$, then the contiguous block to transfer from the donor parent to the offspring is composed from activities 4, 5, and 6 (the

activities in positions 6, 7, and 8 in the donor sequence). Consequently, the positions n_0 and n_3 in the receiver parent are position 3 (where we have activity 4), and 8 (where we have activity 6).

We can now start building the activity sequence for the offspring. We copy, from the receiver, all activities between positions 1 and $n_0 - 1$ (i.e., between position 1 and position 2), as well as the activity in position $n_3 + 1$ (i.e., position 9 which is also the last position in the sequence). Activities of the block are then placed in between the activities. The remaining three activities (2, 8, and 7) are free activities as they are neither predecessors nor successors to any activity in the block. These free activities are then added to the offspring sequence, one by one. Activity 2 can be placed in the offspring sequence either between activities 3 and 4 (just before

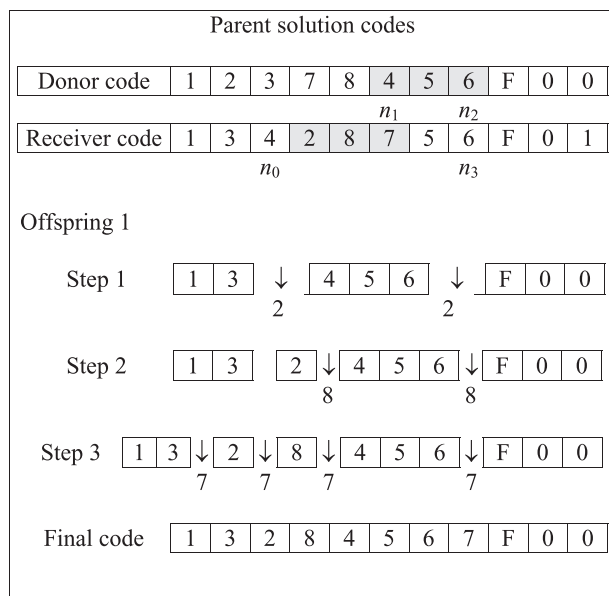


Fig. 8. Application of the proposed two-position crossover.

the block), or between activities 6 and F (just after the block). We assume (randomly) that we decided to place it between activities 3 and 4. Then, as activity 8 is an immediate successor of activity 2, and immediate predecessor of activity F, it can be inserted either between activities 2 and 4, or between 6 and F. Suppose that we (randomly) placed it between activities 2 and 4, then four positions are now possible for activity 7 (see Fig. 8). We randomly place activity 7 between activities 6 and F. Finally, we copy the last two binaries from the donor and we obtain the offspring code {1-3-2-8-4-5-6-7-F-0-0}, as shown in Fig. 8.

Notice that from every pair of parent solutions we generate two offspring solutions. To generate the first offspring, we use the first parent solution as donor and the second as receiver, and to generate the second offspring we consider the second parent as donor and the first one as receiver.

4.5. Mutation

Once the new population has replaced the previous one, a mutation operator is applied to every individual. We randomly select a position, say i , in the activity sequence (i.e., $1 < i < |J|-2$) and draw a random number x_1 from a uniform distribution between 0 and 1. We then switch the jobs in positions i and $i+1$, if both the activity in position i is not an immediate predecessor of the activity in position $i+1$, and $x_1 < \mu$, the mutation probability. We may also mute one or both of the last two binaries in the code. We draw a random number x_2 , and flip the binary to determine whether we use the serial- or parallel-scheme to construct the schedule if $x_2 < \mu$. We then draw another random variable x_3 , and change the last binary of the code if $x_3 < \mu$. In the following numerical tests, we used $\mu = 0.05$.

5. Computational results

The proposed genetic algorithm was implemented in MATLAB release 2014, and tested on a personal computer with Intel Core i7 (2.0 gigahertz) processor, and 6 gigabyte RAM. First, the GA was used to solve the RCPSP with transfer times used by Krüger and Scholl (2009). These test instances are based on instances from the project scheduling instances library, PSPLIB

Table 1
Main characteristics of Krüger and Scholl's (2009) instances.

Set	Network complexity (NC)	Resource factor (RF)	Resource strength (RS)	Total number of instances
J30	[1.5,2.1]	[0.25,1.0]	0.25 0.50 0.75 1.00	120 120 120 120 480
J60	All J30 instances [1.5,2.1]	[0.25,1.0]	0.25 0.50 0.75 1.00	52 108 120 120 400
All instances				880

(<http://www.om-db.wi.tum.de/psplib>). The main characteristics of these problems are presented in Table 1.

Table 2 presents the results obtained by the proposed GA while limiting the number of generated solutions L , to 5000. It is important to note that our GA was coded in MATLAB, which, as reported by Andrews (2012), can be about 500 times slower than C++. Our objective was, however, to get good results rather than quick results. The computation times given are the CPU times until the optimum was obtained, which is similar to what Poppenborg and Knust (2016) reported. In cases where the optimal solution is not reached, the computational time corresponds to the time required to generate 5000 solutions.

Table 3 compares our results to those obtained by the multi-pass heuristic of Krüger and Scholl (2009), to the best results obtained by the GA of Krüger (2009), by the tabu search algorithm presented by Poppenborg and Knust (2014), and the best tabu search results presented by Poppenborg and Knust (2016). Our results were obtained while limiting the number of generated solutions to 5000, while Krüger's were the best of several GA runs, using different parameter sets; the results of the tabu search heuristic reported in Poppenborg and Knust (2014) were obtained with 10,000 visited solutions, and the results reported in Poppenborg and Knust (2016) are the best of several runs of their proposed tabu search.

Table 3 shows that our proposed GA performed well and gave good results, but required longer computation times, largely due to using MATLAB to code the algorithm. The average percentage deviation from the optimum obtained by our GA is 0.42%, while for the tabu search (Poppenborg & Knust, 2016), and the GA (Krüger, 2009) it was 0.82% and 0.26%, respectively. Our GA reached the optimal solution for 782 (88.9%) of the 880 test instances.

Krüger (2009) noted that the instances with RF=1.0 and RS=0.2 were the most difficult to solve. For the 30 instances with 30 activities, RF=1.0 and RS=0.2, the average deviation from the optimum was 1.03% when using the best parameter setting for her GA algorithm, Poppenborg and Knust (2016) reported a deviation of 5.02% for these same 30 instances. Our GA produced an average deviation of 1.60%.

Second, we used our GA to solve the J30, J60, J90, and J120 test instances with randomly generated transfer times solved by Poppenborg and Knust (2016). The numbers of instances in these four groups are: 96, 96, 96, and 120, respectively. The results obtained are presented in Table 4. As we do not have the optimal solutions of these instances, the table gives the average deviation with respect to the lower bound LB_0 suggested by Krüger (2009). Our results are compared to the best results obtained by the different parameter settings of the tabu search heuristic proposed by Poppenborg and Knust (2016). Our GA obtained smaller average percentage deviations for all instance sets. It can also be seen that this deviation decreases with an increase of the resource strength,

Table 2

Results obtained by the proposed genetic algorithm.

Set	Network complexity (NC)	Resource factor (RF)	Resource strength (RS)	Total number of instances	Average deviation from optimum %	Maximum deviation from optimum %	Number of times the optimum was found	Percentage of times the optimum was found	Average CPU time (seconds)
J30	[1.5,2.1]	[0.25,1.0]	0.2	120	0.78	6.12	89	74.17	530.04
			0.50	120	0.20	4.55	110	91.17	56.35
			0.75	120	0.02	1.82	119	98.33	13.77
			1.00	120	0.00	0.00	120	100	0.2
				480	0.25	6.12	438	91.25	139.96
J60	[1.5,2.1]	[0.25,1.0]	0.2	52	3.19	10.91	26	50.00	637.56
			0.50	108	0.81	6.10	78	72.22	423.07
			0.75	120	0.00	0.00	120	100	11.12
			1.00	120	0.00	0.00	120	100	0.41
				400	0.63	12.05	344	86.00	200.57

Table 3

Comparison with published methods.

Set	Number of instances	Performance measure	Multi-pass (Krüger & Scholl, 2009)	Best results GA (Krüger, 2009)	Tabu search (Poppenborg & Knust, 2014)	Best result (Poppenborg & Knust, 2016)	Our GA
J30	480	Percentage deviation from the optimum	1.63	0.16	1.27	0.81	0.25
		Number of times the optimum was obtained	322	459	-	393	438
J60	400	Percentage deviation from the optimum	1.61	0.38	0.98	0.84	0.63
		Number of times the optimum was obtained	284	355	-	337	344
All	880	Percentage deviation from the optimum	1.62	0.26	1.14	0.82	0.42
		Number of times the optimum was obtained	606	814	-	730	782

Table 4Average percentage deviation from the lower bound LB₀.

Set	NC	RF	RS	Number of instances	Best results of Poppenborg and Knust (2016)	Proposed genetic algorithm with				
						1000 Visited solutions	2000	3000	4000	5000
J30	[1.5–2.1]	[0.25–1.00]	0.2	24	76.36	74.96	74.27	73.74	73.51	73.51
			0.5	24	33.28	33.36	33.29	33.18	33.18	33.18
			0.75	24	19.25	19.21	19.04	18.95	18.95	18.89
			1.00	24	3.55	3.89	3.89	3.8	3.72	3.72
			All J30 instances	96	33.11	32.85	32.62	32.42	32.34	32.32
J60	[1.5–2.1]	[0.25–1.00]	0.2	24	100.67	95.64	94.32	93.62	93.24	92.9
			0.5	24	29.86	28.01	27.43	27.08	26.85	26.63
			0.75	24	11.68	10.8	10.69	10.33	10.33	10.22
			1.00	24	0.97	0.87	0.76	0.76	0.7	0.7
			All J60 instances	96	35.79	33.83	33.30	32.95	32.78	32.61
J90	[1.5–2.1]	[0.25–1.00]	0.2	24	107.38	101.86	101.35	100.57	100.19	99.97
			0.5	24	24.58	21.3	20.68	20.41	20.27	20.27
			0.75	24	12.32	8.47	8.13	7.53	7.44	7.4
			1.00	24	0.57	0.37	0.37	0.37	0.33	0.28
			All J90 instances	96	36.21	33.00	32.63	32.22	32.06	31.98
J120	[1.5–2.1]	[0.25–1.00]	0.1	24	188.62	182.27	180.73	180.12	179.12	178.43
			0.2	24	121.91	116.94	116.10	115.33	114.95	114.86
			0.3	24	64.55	60.41	59.32	58.66	58.20	57.92
			0.4	24	43.71	39.05	38.27	37.67	37.35	37.12
			0.5	24	27.55	22.88	22.12	21.94	21.73	21.55
All instances	All J120 instances		120	89.27	84.31	83.31	82.74	82.27	81.97	
			408	50.99	48.25	47.69	47.30	47.06	46.91	

RS, and increases with an increase in the number of project activities. It must be remembered, however, that this deviation is with respect to a lower bound, and not with respect to the optimal solution. The deviation with respect to the optimal project duration is generally much smaller.

To complete our analysis of the proposed GA, Table 5 presents the average improvement of its solutions with respect to the best results obtained by the tabu search heuristic developed by Poppenborg and Knust (2016). The table shows that the GA with

5000 visited solutions gave an average percentage improvement of 2.18%, and that this improvement increased with an increase in the number of project activities. Thus, while the average percentage improvement is 0.37% for the instances with 30 activities, it increases to 3.62% for the 120-activity instances.

Table 6 gives the number of times the GA with 5000 solutions gave a better, equal, or a worse solution compared to the best tabu search solutions. It can be seen that the proposed GA gave a better solution for 220 (54%) of the 408 tested instances, a better

Table 5

Average percentage improvement obtained by the GA with respect to best results reported in Poppenborg and Knust (2016).

Set	Number of instances	Average percentage improvement for the proposed GA with				
		1000 Visited solutions	2000	3000	4000	5000
J30	96	0.03	0.18	0.31	0.36	0.37
J60	96	0.96	1.28	1.52	1.63	1.75
J90	96	1.90	2.17	2.46	2.57	2.62
J120	120	2.32	2.89	3.22	3.46	3.62
All	408	1.36	1.70	1.96	2.09	2.18

Table 6

The results obtained by the GA with 5000 visited solutions compared to the best results reported in Poppenborg and Knust (2016).

Set	Number of instances	Average percentage improvement	Number of instances where the GA gave		
			Better Solution than the best Tabu search solution	Equal	Worse
J30	96	0.37	24	57	15
J60	96	1.75	44	42	10
J90	96	2.62	56	34	6
J120	120	3.62	96	8	16
All	408	2.18	220	141	47

or equal solution for 361 instances (88.5%), and a worse solution for 47 (11.5%) instances.

6. Conclusion

In this paper, the resource-constrained project scheduling problem with sequence dependent transfer times (RCPSPPT) was studied. A GA using a two-point crossover operator was proposed and experimental results showed that this GA is able to efficiently solve the problem. This GA was compared to methods published in the literature using two sets of instances constructed by the PROGEN project generator, to which transfer times were added. The obtained results showed that the proposed GA outperformed most of the previously published methods.

Further research is needed to develop more efficient optimal methods, and more efficient heuristics to solve the RCPSPPT. The development of an optimal solution procedure capable of solving larger and more difficult test instances would allow a more complete assessment of the performance of the proposed GA algorithm and other heuristics to be provided.

Acknowledgments

This research work was partially supported by Grant OPG0036509 from the Canadian Natural Sciences and Engineering Research Council (NSERC). This support is gratefully acknowledged. We also thank Professor Sigrid Knust for providing us with test instances.

References

- Alcaraz, J., Maroto, C., & Ruiz, R. (2003). Solving the multi-mode resource-constrained project scheduling problem with genetic algorithms. *Journal of the Operational Research Society*, 54, 614–626.
- Andrews, T. (2012). *Computation time comparison between Matlab and C++ using launch windows*. California Polytechnic State University. (<http://digitalcommons.calpoly.edu/erosp/78>). Accessed 12 May 2017.

- Baer, T., Brucker, P., & Knust, S. (1999). Tabu search algorithms and lower bounds for the resource-constrained project scheduling problem. In S. Voß, S. Martello, I. Osman, & C. Roucairol (Eds.), *Meta-heuristics* (pp. 1–18). New York: Springer Science+Business Media.
- Bierwirth, C., Mattfeld, D., & Kopfer, H. (1996). On permutation representations for scheduling problems. *Parallel Problem Solving from Nature, IV*, 310–318.
- Blazewicz, J., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1983). Scheduling subject to resource constraints: Classification and complexity. *Discrete Applied Mathematics*, 5, 11–24.
- Bector, F. F. (1990). Some efficient multi-heuristic procedures for resource-constrained project scheduling. *European Journal of Operational Research*, 49, 3–13.
- Bector, F. F. (1996). Resource-constrained project scheduling by simulated annealing. *International Journal of Production Research*, 34, 2335–2351.
- Bouleimen, K., & Lecocq, H. (2003). A new efficient simulated annealing algorithm for the resource-constrained project scheduling problem and its multiple mode version. *European Journal of Operational Research*, 149, 268–281.
- Brooks, G., & White, C. (1965). An algorithm for finding optimal or near optimal solution to the production scheduling problem. *Journal of Industrial Engineering*, 16, 34–40.
- Brucker, P., Knust, S., Schoo, A., & Thiele, O. (1998). A branch and bound algorithm for the resource-constrained project scheduling problem. *European Journal of Operational Research*, 107, 272–288.
- Carlier, J., & Latapie, B. (1991). Une méthode arborescente pour résoudre les problèmes cumulatifs. *RAIRO. Recherche opérationnelle*, 25, 311–340.
- Davis, J. (1985). Job shop scheduling with genetic algorithms. In *Proceedings of the first international conference on genetic algorithms* (pp. 136–140).
- Debels, D., De Reyck, B., Leus, R., & Vanhoucke, M. (2006). A hybrid scatter search/electromagnetism meta-heuristic for project scheduling. *European Journal of Operational Research*, 169, 638–653.
- Demeulemeester, E. L., & Herroelen, W. S. (1996). An efficient optimal solution procedure for the preemptive resource-constrained project scheduling problem. *European Journal of Operational Research*, 90, 334–348.
- Djerid, L., Portman, M. C., & Villon, P. (1996). Performance analysis of permutation cross-over genetic operators. *Journal of Decision Systems*, 4, 131–140.
- Hartmann, S. (1998). A competitive genetic algorithm for resource constrained project scheduling. *Naval Research Logistics*, 45, 733–750.
- Hartmann, S. (2001). Project scheduling with multiple modes: A genetic algorithm. *Annals of Operations Research*, 102, 111–135.
- Hartmann, S. (2002). Project scheduling with multiple modes: A genetic algorithm. *Naval Research Logistics*, 49, 433–448.
- Kelly, J. (1963). The critical-path method: Resources planning and scheduling. In J. F. Muth, & G. L. Thomson (Eds.), *Industrial planning scheduling* (pp. 347–365). New Jersey: Prentice-Hall.
- Kolisch, R. (1996a). Efficient priority rules for the resource-constrained project scheduling problem. *Journal of Operations Management*, 14, 179–192.
- Kolisch, R. (1996b). Serial and parallel resource-constrained project scheduling methods revisited: Theory and computation. *European Journal of Operational Research*, 90, 320–333.
- Kolisch, R., & Drexel, A. (1997). Local search for nonpreemptive multi-mode resource-constrained project scheduling. *IIE Transactions*, 29, 987–999.
- Krüger, D. (2009). *Multi-project scheduling with transfers*. PhD dissertation Germany: University of Jena.
- Krüger, D., & Scholl, A. (2009). A heuristic solution framework for the resource constrained (multi-) project scheduling problem with sequence-dependent transfer times. *European Journal of Operational Research*, 197, 492–508.
- Lova, A., Tormos, P., Cervantes, M., & Barber, F. (2009). An efficient hybrid genetic algorithm for scheduling projects with resource constraints and multiple execution modes. *International Journal of Production Economics*, 117, 302–316.
- Mendes, J. J. D. M., Gonçalves, J. F., & Resende, M. G. (2009). A random key based genetic algorithm for the resource constrained project scheduling problem. *Computers & Operations Research*, 36, 92–109.
- Mobini, M. M., Rabbani, M., Amalnik, M., Razmi, J., & Rahimi-Vahed, A. (2009). Using an enhanced scatter search algorithm for a resource-constrained project scheduling problem. *Soft Computing*, 13, 597–610.
- Nonobe, K., & Ibaraki, T. (2002). Formulation and tabu search algorithm for the resource constrained project scheduling problem. In C. C. Ribeiro, & P. Hansen (Eds.), *Essays and surveys in metaheuristics* (pp. 557–588). New York: Springer Science+Business Media.
- Patterson, J. H., Talbot, B. F., Slowinski, R., & Wegelaz, J. (1990). Computational experience with a backtracking algorithm for solving a general class of precedence and resource-constrained scheduling problems. *European Journal of Operational Research*, 49, 68–79.
- Poppenborg, J., & Knust, S. (2014). A flow-based tabu search algorithm for the RCPSP with transfer times. In *Proceedings of the 14th international conference on project management and scheduling* (pp. 181–184).
- Poppenborg, J., & Knust, S. (2016). A flow-based tabu search algorithm for the RCPSP with transfer times. *OR Spectrum*, 38, 305–335.
- Valls, V., Ballestín, F., & Quintanilla, S. (2004). A population-based approach to the resource-constrained project scheduling problem. *Annals of Operations Research*, 131, 305–324.
- Zamani, R. (2013). A competitive magnet-based genetic algorithm for solving the resource-constrained project scheduling problem. *European Journal of Operational Research*, 229, 552–559.